

# Rapport de benchmark

BenchmarkTick-16 — analyse CPU, mémoire et hyperfine

|                |                                                                        |
|----------------|------------------------------------------------------------------------|
| Généré le      | 2026-09-24T08:27:56.2825059Z                                           |
| Commande       | go test ./game -run '^\$' -bench '^BenchmarkTick\$' -benchmem -count 1 |
| Plateforme     | windows/amd64                                                          |
| Processeur     | AMD Ryzen 7 7735HS with Radeon Graphics                                |
| Profil CPU     | cpu-20260924-082756401.prof                                            |
| Profil mémoire | memory-20260924-082756401.prof                                         |

## 1. Résultats du benchmark

| Benchmark        | Moyenne   | ns/op     | B/op | allocs/op |
|------------------|-----------|-----------|------|-----------|
| BenchmarkTick-16 | 131.68 ms | 131682288 | 0    | 0         |

### Détail hyperfine

| Métrique          | Valeur   |
|-------------------|----------|
| Moyenne           | 2.5553 s |
| Écart-type        | n/a      |
| Médiane           | 2.5553 s |
| Temps utilisateur | 1.4941 s |
| Temps système     | 0.5737 s |
| Min               | 2.5553 s |
| Max               | 2.5553 s |

### Résumé CPU et mémoire

| Mesure                    | Valeur     |
|---------------------------|------------|
| CPU échantillonné (pprof) | 1220.00ms  |
| Mémoire allouée (pprof)   | 19630.47kB |
| Mémoire par opération     | 0 B/op     |

### Garbage collector Go

|                 |           |
|-----------------|-----------|
| Cycles GC       | 57        |
| Temps GC cumulé | 56.530 ms |

|                          |            |
|--------------------------|------------|
| Plus longue pause        | 2.700 ms   |
| Temps CPU GC cumulé      | 315.970 ms |
| Objectif mémoire maximal | 97.00 MB   |

## 2. Profil CPU — où passe le temps ?

Type de profil : **cpu**. Total échantillonné : **1220.00ms**.

| Fonction                                       | Flat   | Flat % | Cumulé | Cumulé % |
|------------------------------------------------|--------|--------|--------|----------|
| jeu-de-la-vie/game.(*Board).shouldInfect       | 1010ms | 82.79% | 1200ms | 98.36%   |
| jeu-de-la-vie/game.(*Board).index (inline)     | 110ms  | 9.02%  | 110ms  | 9.02%    |
| jeu-de-la-vie/game.squaredDistance (inline)    | 60ms   | 4.92%  | 60ms   | 4.92%    |
| math/rand.(*Rand).Int63 (inline)               | 20ms   | 1.64%  | 20ms   | 1.64%    |
| runtime.cgocall                                | 20ms   | 1.64%  | 20ms   | 1.64%    |
| internal/syscall/windows/registry.Key.GetMUISt | 0      | 0.00%  | 20ms   | 1.64%    |
| internal/syscall/windows/registry.regLoadMUISt | 0      | 0.00%  | 20ms   | 1.64%    |
| jeu-de-la-vie/game.(*Board).Step               | 0      | 0.00%  | 1200ms | 98.36%   |

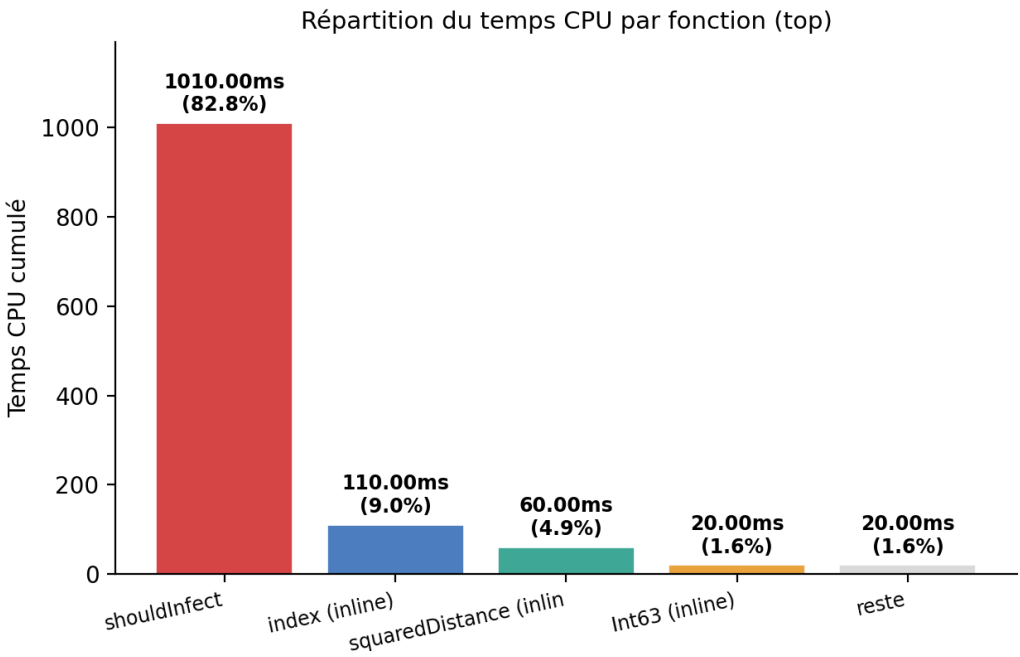


Figure — Répartition du temps CPU par fonction (temps flat, top fonctions).

File: game.test.exe  
 Build ID: C:\Users\sadou\JeuDeLaVie\game.test.exe2026-09-24 10:27:34.4442249 +0200 CEST  
 Type: cpu  
 Time: 2026-09-24 10:27:56 CEST  
 Duration: 1.23s, Total samples = 1220ms (99.12%)  
 Showing nodes accounting for 1220ms, 100% of 1220ms total  
 Showing top 15 nodes out of 32  
 See <https://git.io/JYMW> for how to read the graph

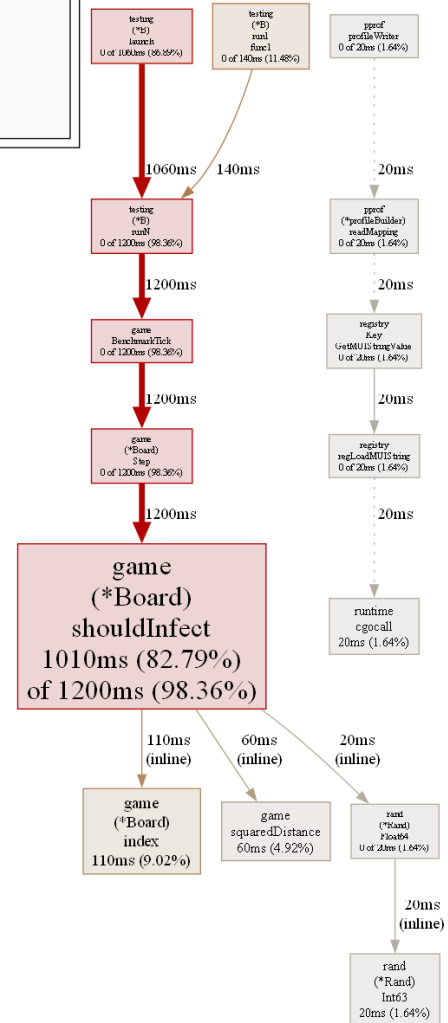


Figure — Graphe d'appels CPU (go tool pprof -png).

## 2.1 Diagnostic automatique

La fonction dominante de ce run est **jeu-de-la-vie/game.(\*Board).shouldInfect**, responsable de **82.8%** du temps CPU total (flat), pour **98.4%** cumulé avec ses appels. C'est la cible prioritaire d'optimisation pour ce run.

- Le nom de cette fonction suggère une recherche/un parcours répété (distance, voisinage, scan...) — vérifier si elle itère sur l'ensemble des données à chaque appel plutôt que sur un sous-ensemble borné (structure spatiale : grille, quadtree, index).
- Comparer ce hotspot avec le run précédent : s'il a changé de fonction, l'optimisation précédente a fonctionné et un nouveau goulot est apparu ailleurs.
- Vérifier s'il existe un algorithme en  $O(n)$  ou  $O(\log n)$  équivalent pour cette opération avant d'optimiser au niveau micro (inlining, allocations).

Rappel — pourquoi la complexité algorithmique domine  
à mesure que les données grandissent (schéma, non mesuré)

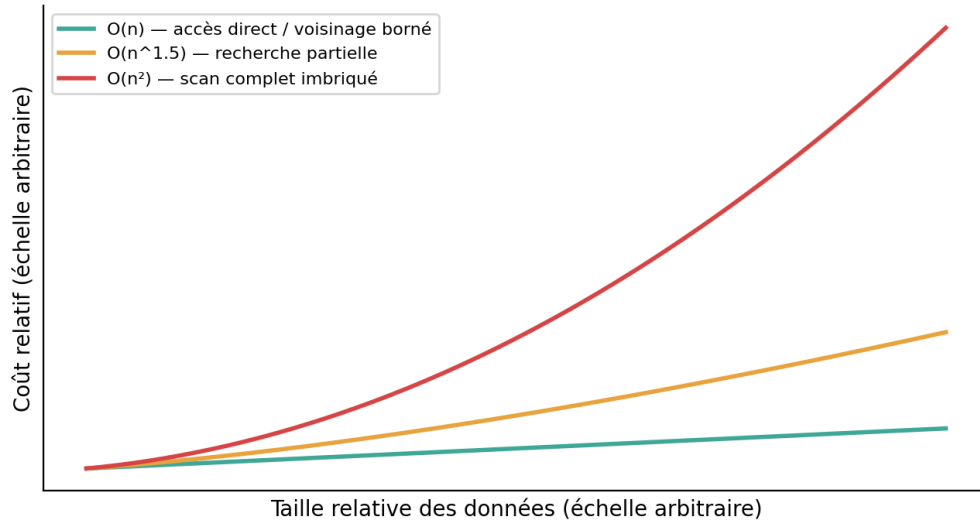


Figure — Rappel pédagogique sur la complexité algorithmique (schéma générique).

### 3. Profil mémoire — allocations

Type de profil : `alloc_space`. Total : **19630.47kB**.

| Fonction                             | Flat      | Flat % | Cumulé     | Cumulé % |
|--------------------------------------|-----------|--------|------------|----------|
| jeu-de-la-vie/game.NewBoard (inline) | 7583.18kB | 38.63% | 7583.18kB  | 38.63%   |
| jeu-de-la-vie/game.RandomBoard       | 5655.11kB | 28.81% | 13238.29kB | 67.44%   |
| runtime.mallocgc                     | 2052kB    | 10.45% | 2052kB     | 10.45%   |
| runtime/pprof.StartCPUProfile        | 1762.94kB | 8.98%  | 1762.94kB  | 8.98%    |
| compress/flate.NewWriter (inline)    | 902.59kB  | 4.60%  | 1553.21kB  | 7.91%    |
| compress/flate.(*compressor).init    | 650.62kB  | 3.31%  | 650.62kB   | 3.31%    |
| testing.(*B).ResetTimer              | 512.02kB  | 2.61%  | 512.02kB   | 2.61%    |
| syscall.LoadDLL                      | 512.01kB  | 2.61%  | 512.01kB   | 2.61%    |

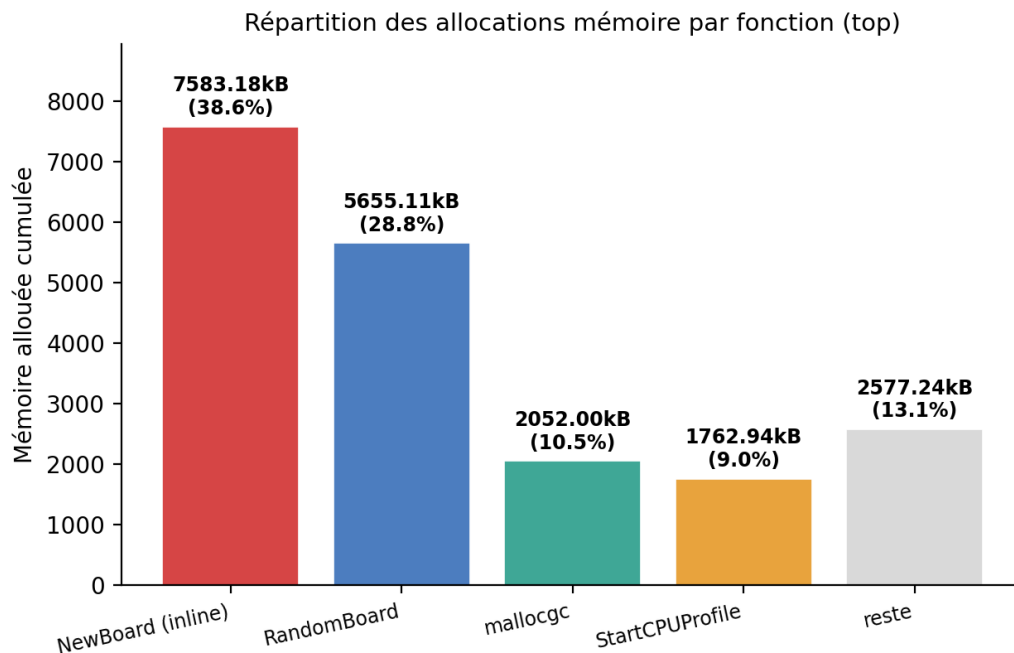


Figure — Répartition des allocations mémoire par fonction.

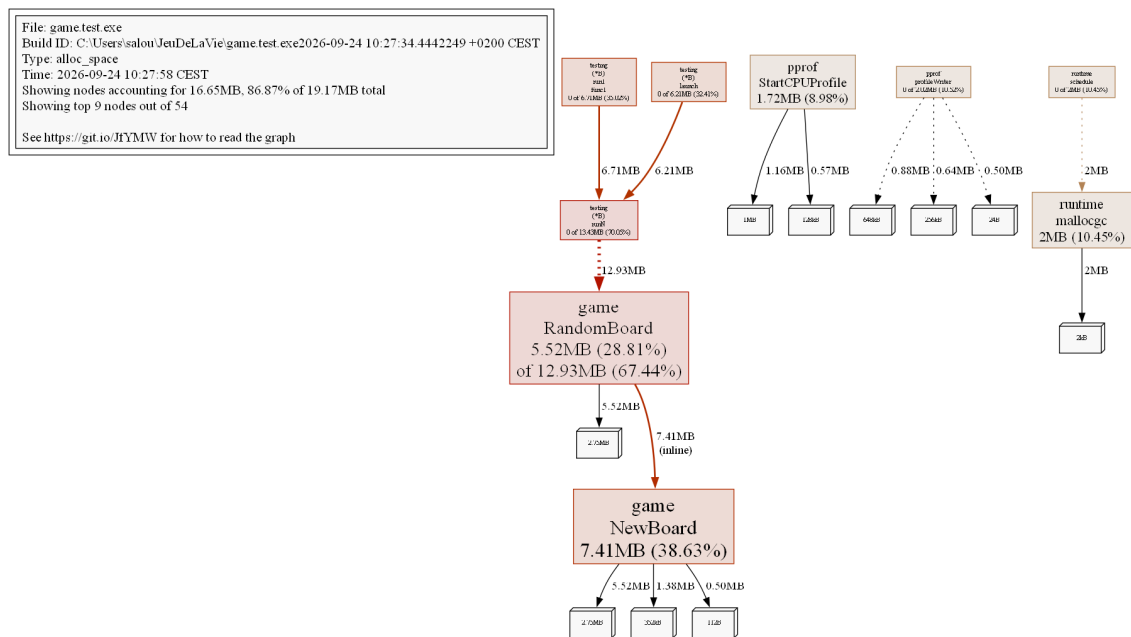


Figure — Graphe d'appels mémoire (go tool pprof -png).

## 4. Limites de ce benchmark & informations manquantes

Générées automatiquement à partir de la configuration détectée :

- **Peu de runs (1).** Avec moins de 5 exécutions, aucune médiane/écart-type fiable n'est disponible. Recommandé :  $\geq 10$  runs (-count=10 ou hyperfine --min-runs 10).
- **Aucun warmup.** La première exécution peut être biaisée (cache froid, JIT du GC non stabilisé). Ajouter 1-2 warmups avant la mesure.
- **Pas de comparaison avant/après** intégrée à ce rapport — conservez les PDF successifs pour comparer les hotspots et le temps moyen dans le temps.
- **Un seul point de taille de données.** Si applicable, mesurer plusieurs tailles permettrait de vérifier empiriquement la complexité algorithmique observée en section 2.

### Notes de l'outil de benchmark

- Mesure du calcul du tick sur une carte  $600 \times 600$ .
- La carte et le générateur aléatoire utilisent toujours la seed 42.
- La latence API est mesurée à part dans le frontend, sans rendu canvas.
- La trace GC détaillée est enregistrée dans le fichier `gc-\*.log` et résumée dans le PDF.
- Les métriques I/O, réseau et base de données ne sont pas instrumentées dans ce benchmark.

---

Rapport généré automatiquement le 2026-09-24 10:28:05 par generate\_benchmark\_report.py, à partir des fichiers les plus récents du dossier de benchmarks. Analyse réalisée avec go tool pprof.